

Flask CI/CD Pipeline Using Jenkins, Docker, AWS ECR and EC2

Project Summary

This project demonstrates the implementation of a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Flask web application using Jenkins, Docker, Amazon Elastic Container Registry (ECR), and Amazon EC2.

The primary objective was to automate the software delivery process from source code commit to deployment, following industry-standard DevOps practices.

Objectives

- Automate application build process.
 - Containerize the application using Docker.
 - Store Docker images in Amazon ECR.
 - Deploy containerized applications on AWS EC2.
 - Implement CI/CD using Jenkins.
 - Gain hands-on experience with AWS and DevOps tools.
-

Tools and Technologies

Tool	Purpose
GitHub	Source Code Management
Jenkins	CI/CD Automation
Docker	Containerization
AWS ECR	Container Registry
AWS EC2	Application Hosting
Python Flask	Web Application
Ubuntu Linux	Operating System

Project Architecture

Developer pushes code to GitHub.

GitHub Repository → Jenkins Pipeline → Docker Build → Amazon ECR → AWS EC2 → Flask Container → Web Application

Implementation Steps

Step 1: Create Flask Application

A simple Flask web application was created using Python.

Files:

- app.py
 - requirements.txt
-

Step 2: Dockerize the Application

A Dockerfile was created to package the application into a portable container image.

Commands:

```
docker build -t flask-app .
```

```
docker run -d -p 5000:5000 flask-app
```

Step 3: Configure Jenkins

Jenkins was deployed inside a Docker container.

Required plugins:

- Git
 - Pipeline
 - Docker Pipeline
-

Step 4: Create Jenkins Pipeline

Pipeline stages:

1. Build Docker Image
2. Tag Docker Image
3. Login to Amazon ECR

4. Push Docker Image to ECR

Step 5: Configure AWS ECR

Created a private ECR repository:

flask-devops-app

Configured AWS credentials inside Jenkins.

Step 6: Deploy on EC2

Docker image was pulled from ECR and deployed on EC2.

Command:

```
docker run -d -p 5000:5000 432839260003.dkr.ecr.ap-south-2.amazonaws.com/flask-devops-app:latest
```

Challenges Faced

Jenkinsfile Syntax Error

Problem: Pipeline failed due to Markdown backticks accidentally included in Jenkinsfile.

Solution: Removed Markdown formatting and corrected Groovy syntax.

Docker Not Available in Jenkins

Problem: Docker commands were unavailable inside Jenkins container.

Solution: Installed Docker CLI and mounted Docker socket.

Docker Permission Issues

Problem: Jenkins could not communicate with Docker daemon.

Solution:

```
sudo chmod 666 /var/run/docker.sock
```

EC2 Memory Constraints

Problem: Free Tier EC2 instance frequently became unstable.

Solution: Created a 1 GB swap file.

Commands:

```
sudo fallocate -l 1G /swapfile
```

```
sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

Results

- Successfully built Docker images using Jenkins.
 - Successfully pushed images to Amazon ECR.
 - Successfully deployed application on AWS EC2.
 - Verified application accessibility through browser.
 - Implemented end-to-end CI/CD pipeline.
-

Key Learnings

- Jenkins Declarative Pipelines
 - Docker Image Lifecycle
 - AWS ECR Integration
 - Container Deployment
 - Linux Administration
 - CI/CD Automation
 - Troubleshooting Production Issues
-

Future Enhancements

- SonarQube Integration
- Trivy Security Scanning
- Terraform Infrastructure as Code
- Kubernetes Deployment
- Prometheus Monitoring
- Grafana Dashboards

Conclusion

The project successfully demonstrates a complete CI/CD workflow using modern DevOps tools and AWS services. It provides practical experience in automation, containerization, cloud deployment, and infrastructure management while following industry best practices.

Author: Mohammed Amaan Ahmed

DevOps Engineer | AWS | Docker | Jenkins | CI/CD